

Linear, BatchNorm, Dropout

Blumenau Mark, MDS

What do we want?

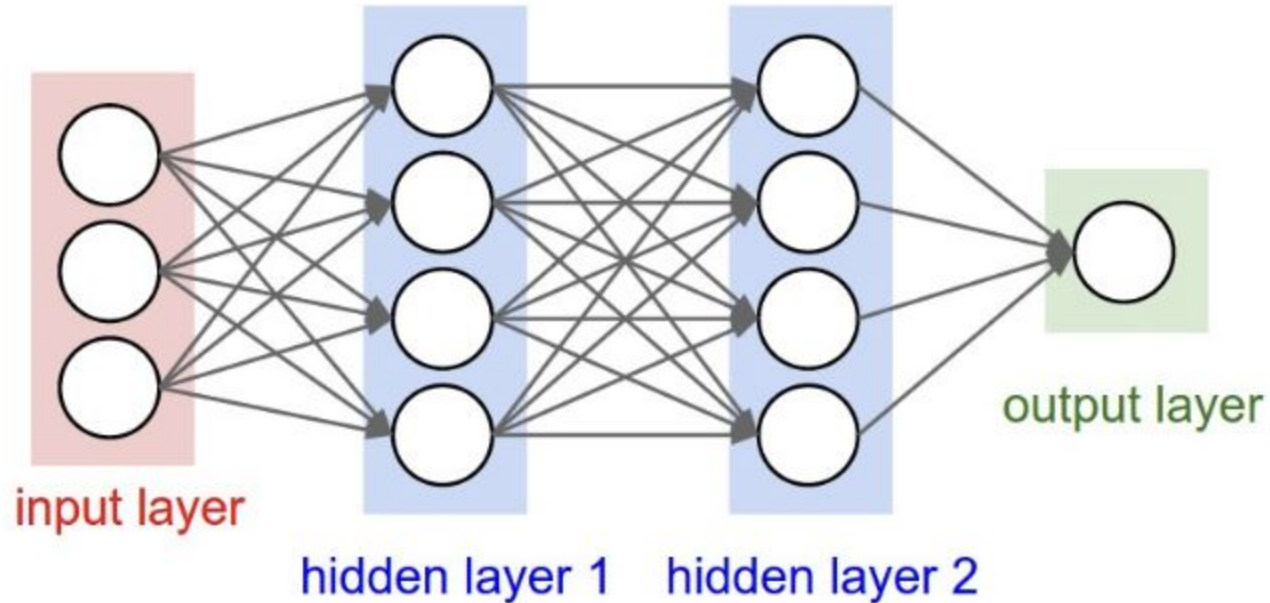
Suppose we have some kind of loss, e. g. MSE:

$$L(y_i, a(x_i)) = (a(x_i, w) - y_i)^2$$

We want:

$$\frac{1}{\ell} \sum_{i=1}^{\ell} L(y_i, a(x_i)) \rightarrow \min_a$$

Fully-connected (=Linear PyTorch) layer



What is inside of it?

Each layer is a set of linear models

$$z_j = \sum_{i=1}^n w_{ji} x_i + b_j$$

Usually there are several layers one after another. How many parameters we have?

(Inputs (=amount of “x”) + 1) * outputs (=amount of “z”), which we set

For n inputs and m outputs: $m(n+1)$

Great! Let's take 2 Linear layers

$$\begin{aligned} s_k &= \sum_{j=1}^m v_{kj} z_j + c_k = \sum_{j=1}^m v_{kj} \sum_{i=1}^n w_{ji} x_i + \sum_{j=1}^m v_{kj} b_j + c_k = \\ &= \sum_{j=1}^m \left(\sum_{i=1}^n v_{kj} w_{ji} x_i + v_{kj} b_j + \frac{1}{m} c_k \right) \end{aligned}$$

Looks like we still have one layer

Same but in matrix form

$$y = xA^T + b$$

$$\begin{aligned} z &= yC^T + d = (xA^T + b)C^T + d = \\ &= xA^T C^T + bC^T + d = \\ &= x(A^T C^T) + (bC^T + d) \end{aligned}$$

$$\hat{A} = A^T C^T$$

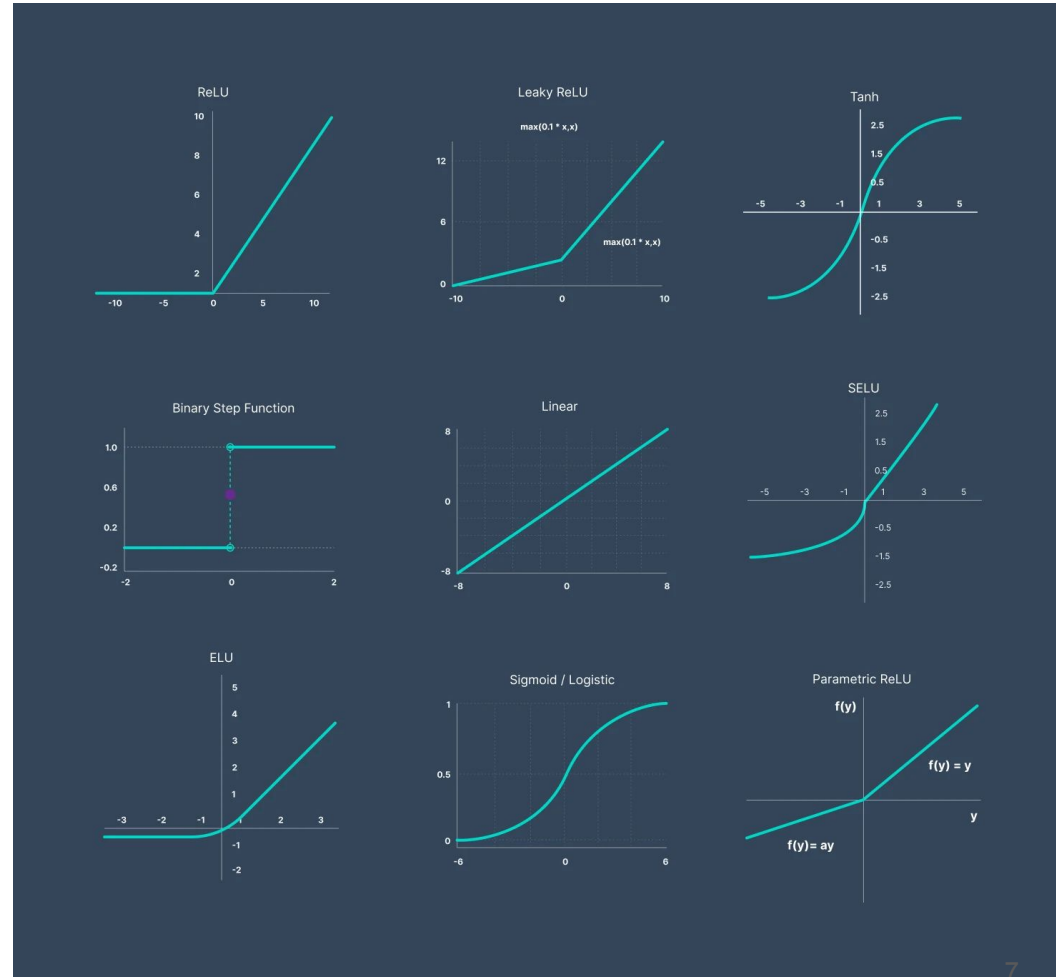
$$\hat{b} = bC^T + d$$

How can we fix that?

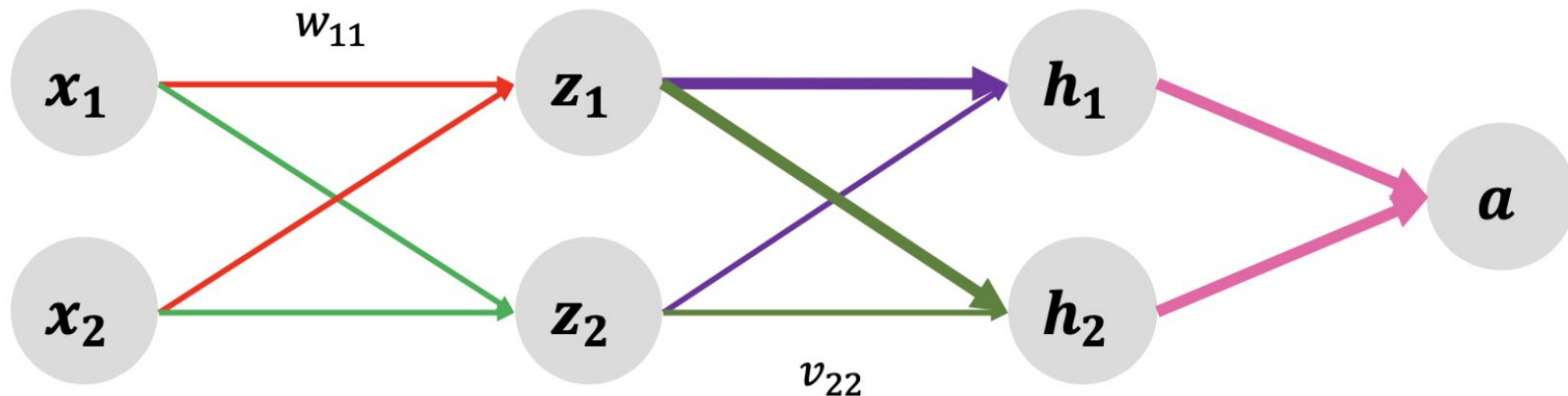
Nonlinearity!

What we look for:

- 1) Continuous differentiability
- 2) Monotonicity
- 3) Finiteness
- 4) Close to linear near the origin



Backpropagation



$$\frac{\partial}{\partial w_j} (a(x_i, w) - y_i)^2 = 2(a(x_i, w) - y_i) \frac{\partial}{\partial w_j} a(x_i, w)$$

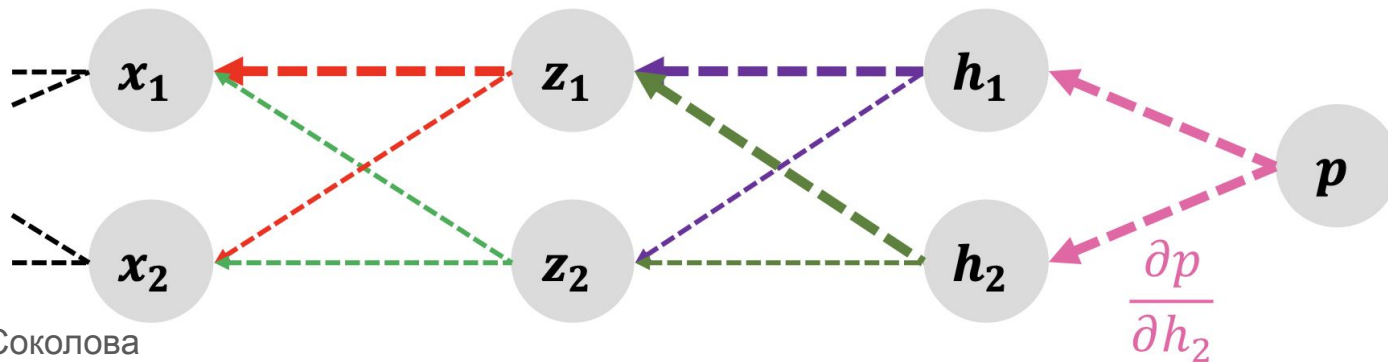
$$\frac{\partial a}{\partial w_{11}} = \frac{\partial a}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_{11}} + \frac{\partial a}{\partial h_2} \frac{\partial h_2}{\partial z_1} \frac{\partial z_1}{\partial w_{11}}$$

$$3: \frac{\partial p}{\partial h_1} \quad \frac{\partial p}{\partial h_2}$$

$$2: \frac{\partial p}{\partial z_1} = \frac{\partial p}{\partial h_1} \frac{\partial h_1}{\partial z_1} + \frac{\partial p}{\partial h_2} \frac{\partial h_2}{\partial z_1} \quad \frac{\partial p}{\partial z_2} = \frac{\partial p}{\partial h_1} \frac{\partial h_1}{\partial z_2} + \frac{\partial p}{\partial h_2} \frac{\partial h_2}{\partial z_2}$$

$$1: \frac{\partial p}{\partial x_1} = \frac{\partial p}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial x_1} + \frac{\partial p}{\partial h_2} \frac{\partial h_2}{\partial z_1} \frac{\partial z_1}{\partial x_1} + \frac{\partial p}{\partial h_1} \frac{\partial h_1}{\partial z_2} \frac{\partial z_2}{\partial x_1} + \frac{\partial p}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial x_1}$$

$$\frac{\partial p}{\partial x_2} = \frac{\partial p}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial x_2} + \frac{\partial p}{\partial h_2} \frac{\partial h_2}{\partial z_1} \frac{\partial z_1}{\partial x_2} + \frac{\partial p}{\partial h_1} \frac{\partial h_1}{\partial z_2} \frac{\partial z_2}{\partial x_2} + \frac{\partial p}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial x_2}$$



Cybenko's theorem (universal approximation theorem)

In free wording

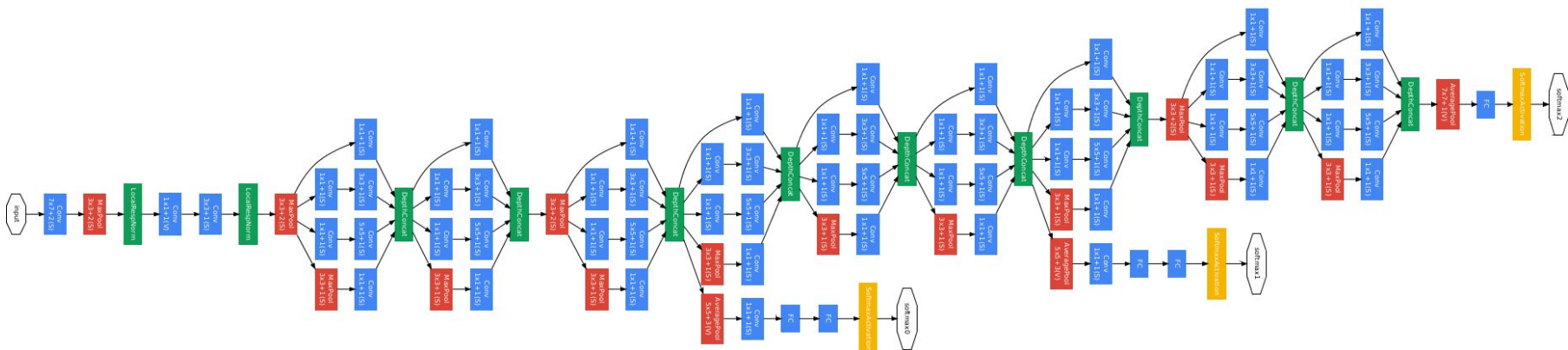
Given a continuous function $f(x)$, we can construct a two-layer neural network that approximates $f(x)$ with any predetermined accuracy.

=> Fully connected layers are a very powerful tool.

What is the catch?

What do we do?

We go deep, very deep. Typical NN:

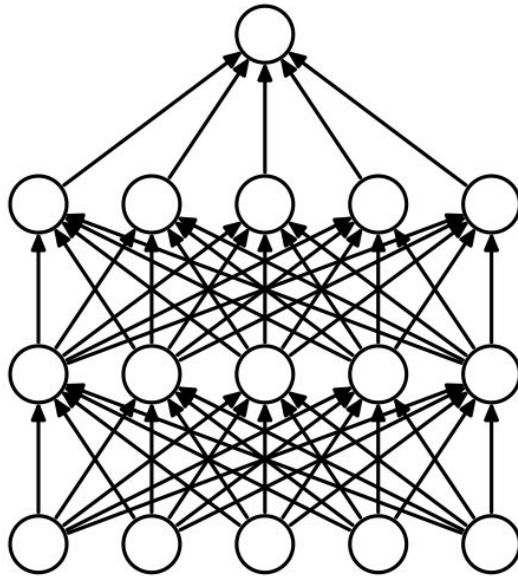


<https://medium.com/swlh/understanding-inception-simplifying-the-network-architecture-54cd31d38949>

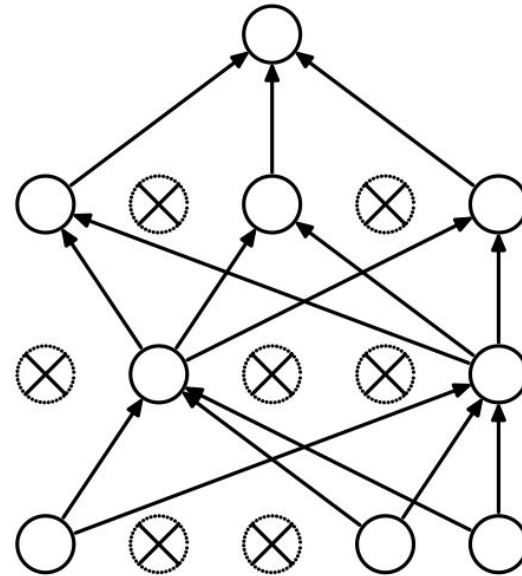
How do we stop overfitting?

1) Weight_decay, see last week

2)



(a) Standard Neural Net



(b) After applying dropout.

How does it work? (PyTorch)

Train mode

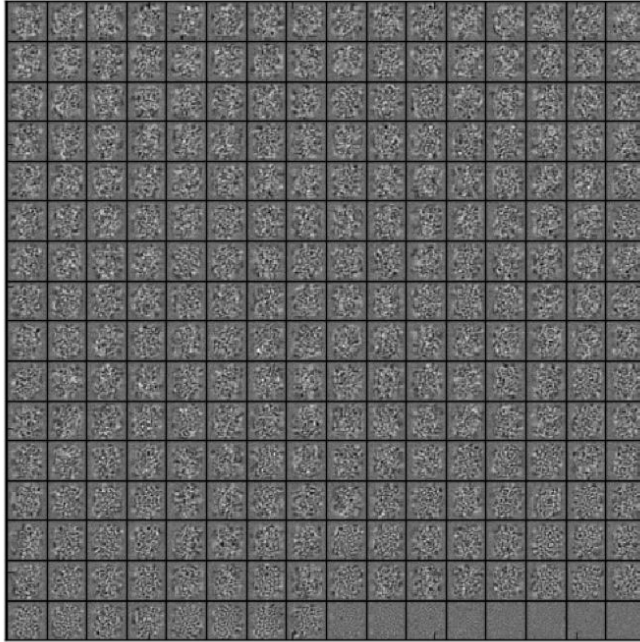
- 1) Each output is set to zero with a probability p (coinflip)
- 2) After that we multiply everything by $1/(1-p)$

Eval mode

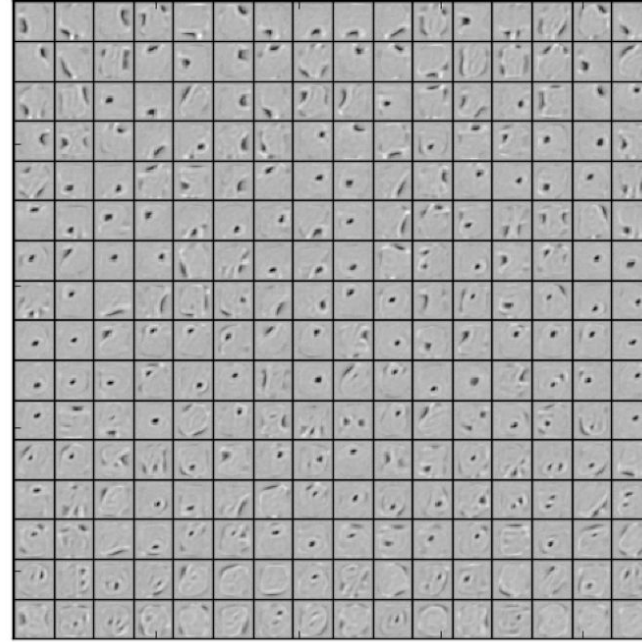
- 1) Do nothing (multiply everything by 1)

Any ideas why?

One can literally see the effect

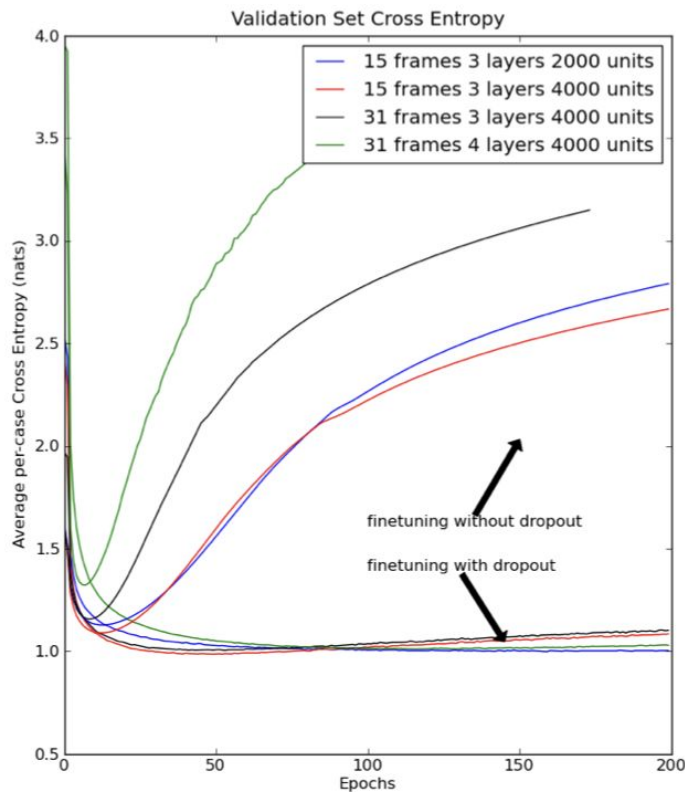
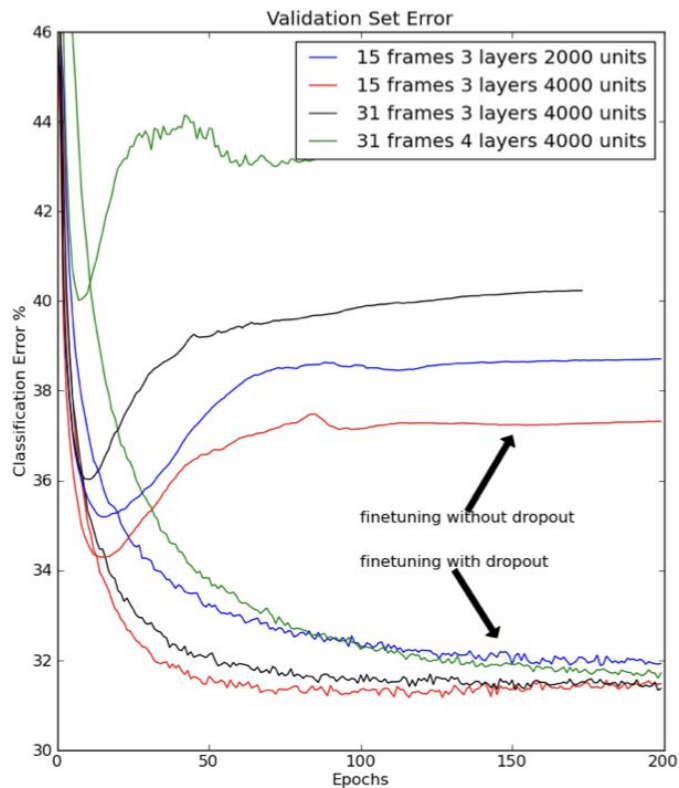


(a) Without dropout



(b) Dropout with $p = 0.5$.

It does work on finetuning



Is that all we can do? BatchNorm

Batch Normalization

$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

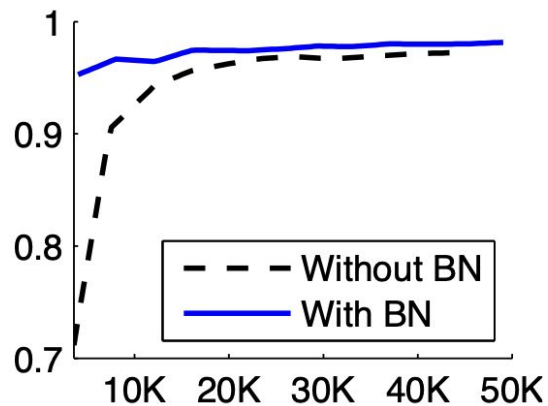
Train mode

Calculate $\mathbf{E}[x]$ and $\mathbf{Var}[x]$

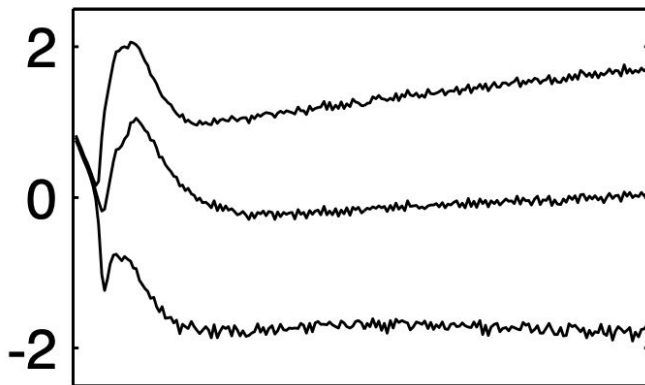
Eval mode

Use moving averages from training with momentum

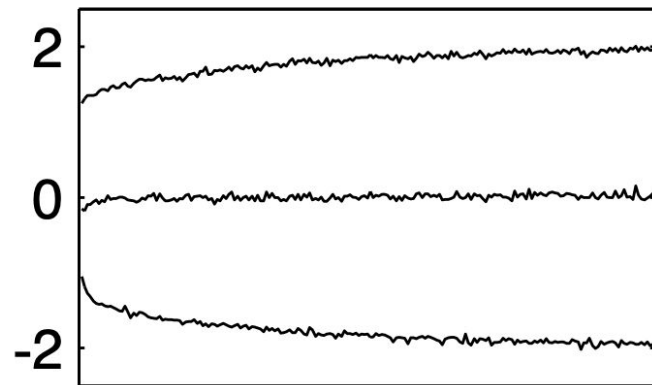
The effect



(a)



(b) Without BN



(c) With BN

Typical loss functions

MSE

$$l_n = (x_n - y_n)^2$$

CrossEntropy

$$l_n = -w_{y_n} \log \frac{\exp(x_{n,y_n})}{\sum_{c=1}^C \exp(x_{n,c})} \cdot 1\{y_n \neq \text{ignore_index}\}$$

Honestly, you can use whatever you feel like

$$\mathbf{e}_1 = \left(\partial_t \hat{\mathbf{v}} + (\hat{\mathbf{v}} \cdot \nabla) \hat{\mathbf{v}} + \nabla \hat{p} + e^{\lambda_1} \hat{\mathbf{v}} - e^{\lambda_2} \nabla^2 \hat{\mathbf{v}} - \hat{\mathbf{f}} \right) L/U^2,$$

$$e_2 = (\partial_x \hat{v}_x + \partial_y \hat{v}_y) L/U,$$

$$e_3 = \left(\partial_x \hat{f}_x + \partial_y \hat{f}_y \right) L^2/U^2.$$

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}} U^2} \sum_{n=1}^{N_{\text{data}}} |\hat{\mathbf{v}}(\mathbf{r}_n^d, t_n^d) - \mathbf{v}^d(\mathbf{r}_n^d, t_n^d)|^2$$

$$\mathcal{L}_{\text{eq}} = \frac{1}{N_{\text{eq}}} \sum_{n=1}^{N_{\text{eq}}} [\mathbf{e}_1^2(\mathbf{r}_n^e, t_n^e) + e_2^2(\mathbf{r}_n^e, t_n^e) + e_3^2(\mathbf{r}_n^e)]$$

Weight initialization

Xavier Glorot

$$W \sim U \left[-\frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}, \frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}} \right]$$

<https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>

Kaiming He

$$W \sim \mathcal{N} \left(0, \frac{2}{n^l} \right)$$

<https://paperswithcode.com/paper/delving-deep-into-rectifiers-surpassing-human>

Why do we care?

